1. Data modeling:
    a. User:
        i. Identification: ID, name, profile pic (?)
        ii. <Location>: lat, long, lastSeen timestamp
        iii. Privacy: public, friends only, private
        iv. Music snapshot: playingNow, title, artist, album, preview
        v. (Compatibility: top genres, top tracks, top artists (match score))
    b. Track:
        i. Title
        ii. Artist
        iii. Album
        iv. isNowPlaying
        v. previewUrl
    c. Artist:
        i. Name
        ii. Listeners?
    d. Friendship (connection between users):
        i. UserID
        ii. FriendID
        iii. Status (pending, accepted,blocked)
        iv. Match score?
    e. Location:
        i. UserID
        ii. Lat
        iii. Long
        iv. Last seen timestamp
    f. What else…
2. Music provider interface:
    a. getPlayingNow(userID): returns current track
    b. ferRecentTracks(userID): history and matching

    c. getPreviewUrl(trackID): for iTunes preview
    d. What else…
3. Map and interaction logic:
    a. Discovery radius (500m - 5km?)
    b. Filtering
    c. Zoom behavior
4. Social:
    a. “Ping”: temporary chat
    b. Friend request
    c. Matching?
    d. Chat itself
    e. Temporary chat should autoclean after expiration (delete everything related from db)
    f. No photos, just text for the beginning, since photos take up a lot of space
5. Technologies
    a. Front: simple kotlin compose and make prettier with figma
    b. Map engine: maplibre, map tiler, open street map
    c. Backend: Supabase (parola normala)
    d. Auth: auth din supabase
    e. DB: postgre din supabase
    f. Storage: tot din supabase
6. Logic:
    a. Now playing sync:

        i. When does it change? Pooling every n seconds or just when the song changes? Use supabase realtime to save battery and resources
        ii. If the user is on “invisible” mode, the data is fetched to their own UI, but never sent to the backend
        iii. Uses data: track, artist, …, timestamp (if user not live)

    b. Matching algo:
        i. Simple: count common artist + tracks
        ii. Advanced: assign weights, niche brings a higher score than mainstream

    c. Map interaction:
        i. Discovery: tap on user icon on the map
        ii. Preview: user can preview what other users are listening
        iii. Ping: start a temporary chat
        iv. Expiration: chat expires after 24h if users are not friends

7. Table definitions (there should already be some auth.users in supabase):

    a. Users (public.users):
        i. Id (PK, FK in auth.users)
        ii. Username (unique)
        iii. Email (optional)
        iv. Lastfm_username (unique)
        v. Avatar_url (optional)
        vi. Privacy_mode (text)
        vii. Created_at (timestamp)

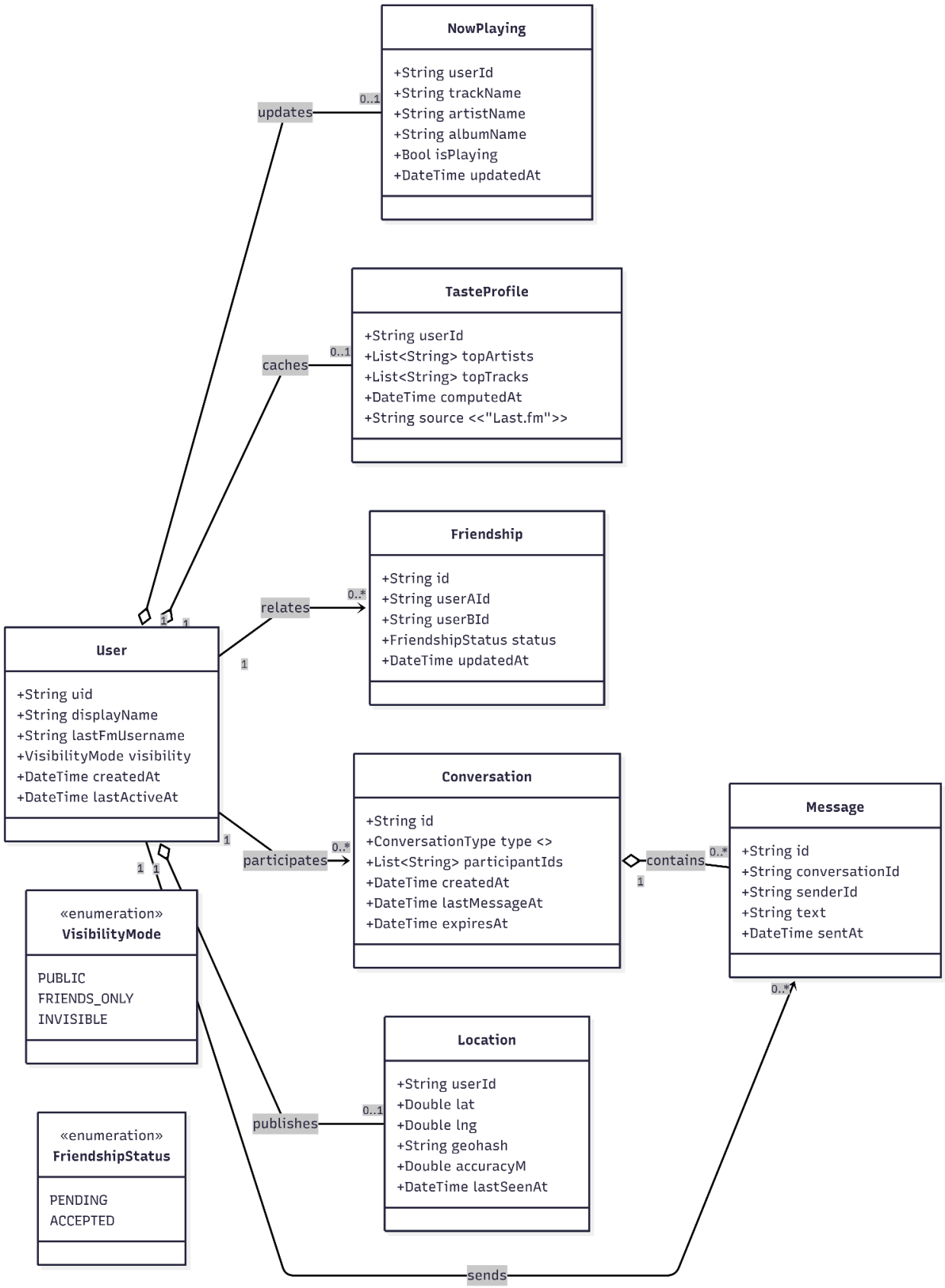
    b. Location:
        i. User_id (PK, FK in public.users)
        ii. Lat (double)
        iii. Lng (double)
        iv. Geohash (? text)
        v. Accuracy (optional int)
        vi. Last_seen_at (timestamp)

    c. CurrentTrack:
        i. User_id (PK, FK in public.users)
        ii. Track_name (text)

        iii. Artist_name (text)
        iv. Album_art (text)
        v. Itunes_preview_url (text, nullable)
        vi. Is_playing (bool)
        vii. Synced_at (timestamp)

    d. Friendship:
        i. User_id (FK)
        ii. Friend_id (FK)
        iii. Status (enum / text)
        iv. Match_score (int, nullable)
        v. Updated_at (? timestamp)
    e. Conversations:
        i. Chat_id (PK)
        ii. User_one_id (FK in public.users)
        iii. User_two_id (FK in public.users)
        iv. Is_persistent (bool)
        v. Expires_at (not() + 24h / null)
        vi. Initial_track_title (text)
        vii. Initial_track_artist (text)
        viii. Last_message_at (timestampt), used for sorting user chats
    f. Messages:
        i. Id (PK)
        ii. Covnersation_id (FK in conversations)
        iii. Sender_id (FK in public.users)
        iv. Content (text)
        v. Sent_at (timestamp)

8. Package structure (each with data, domain, presentation):
    a. Core (for common features)
    b. Auth
    c. Map
    d. Music
    e. Social


NowPlaying
+String userId
+String trackName
+String artistName
+String albumName
+Bool isPlaying
+DateTime updatedAt
updates
0..1
TasteProfile
+String userId
+List<String> topArtists
+List<String> topTracks
+DateTime computedAt
+String source <<"Last.fm">>
caches
0..1
Friendship
+String id
+String userAId
+String userBId
+FriendshipStatus status
+DateTime updatedAt
relates
0..*
1
1
1
User
+String uid
+String displayName
+String lastFmUsername
+VisibilityMode visibility
+DateTime createdAt
+DateTime lastActiveAt
1
participates
0..*
Conversation
+String id
+ConversationType type <>
+List<String> participantIds
+DateTime createdAt
+DateTime lastMessageAt
+DateTime expiresAt
contains
0..*
1
Message
+String id
+String conversationId
+String senderId
+String text
+DateTime sentAt
0..*
1
1
«enumeration»
VisibilityMode
PUBLIC
FRIENDS_ONLY
INVISIBLE
Location
+String userId
+Double lat
+Double lng
+String geohash
+Double accuracyM
+DateTime lastSeenAt
publishes
0..1
«enumeration»
FriendshipStatus
PENDING
ACCEPTED
sends

9. Roadmap

    a. Identity and music link
        i. Set up supabase project (auth and db)
        ii. Implement last.fm auth (one field to link last.fm username to supabase user id)

    b. Sync engine
        i. Build poolling service that calls last.fm every n seconds while app is in foreground
        ii. Implement presence publisher (update current / last track and location in db)
        iii. Implement privacy modes !!! if invisible is enabled, location is not sent to the db !!! it will appear where it was last seen

    c. Map integration
        i. Set up map libre
        ii. Fetch “nearby users” from supabase and display them (how will I mock other users?)
        iii. Implement zoom interaction (clustering)

    d. iTunes preview

        i. Fetch the preview for the specific track from iTunes API

        ii. Create the UI for the preview

    e. Pings and chat

        i. Implement the temporary chat (marked by an expires at timestamp in the conversations table)

        ii. Add friend requests -> persistent chats

        iii. Matching algo

    f. Prep for demo

        i. Create the users in a csv first

        ii. Create mock users db

        iii. Simulation service and flow (accept request in db)